

GPT-Guided DAgger for VLA

Learning from Executable Code-as-Policy Corrections

Research Proposal III · Zimo Wen

September 2026

Abstract

Vision–language–action policies can execute efficiently but accumulate errors when their own actions lead to states absent from the demonstration distribution. A GPT-based robot agent can reason about such failures and synthesize a recovery program, yet using that agent on every deployment episode retains substantial inference cost. This proposal studies GPT as an approximate corrective expert within a DAgger-style learning loop. The VLA visits states, GPT inspects permitted observations, Code as Policy turns corrections into executable behavior, and aligned corrective actions are aggregated to train the VLA. The research compares dense correction, bounded expert intervention, and budgeted selective queries. Its central criterion is improved performance of the frozen VLA after GPT has been removed. The intended paper investigates the quality, distribution, and cost of executable supervision, rather than a new low-level controller or a permanently assisted agent.

1. Motivation and Standalone Research Hypothesis

Why the VLA needs corrections on its own trajectories. Behavioral cloning learns from demonstrations, but a deployed policy can create small grasp, placement, or sequencing errors that compound into unfamiliar states. Additional successful demonstrations from clean initial states may not cover those states. DAgger addresses this distribution mismatch by querying an expert on states induced by the learner and aggregating the resulting labels. Here, GPT is the proposed expert; DAgger is the data-collection and learning procedure, not the name of a GPT control mode.

What Code as Policy contributes. A natural-language criticism such as “the grasp is unstable; approach from the side” is not an action target for a VLA. Robot-RSI can convert grounded observations and proposed recoveries into programs over executable skills. This makes Code as Policy a candidate bridge from GPT’s correction to time-aligned action supervision. Existing action and controller interfaces are retained. The research contribution concerns how corrections become training data and improve the student, rather than adding force or torque outputs.

A motivating example. A VLA closes its gripper beside a cup and starts lifting empty fingers. GPT sees the current images, robot state, and visible execution history, identifies the failed grasp, and produces a short re-observe–reapproach–grasp program. Execution records the observations and actual commands during recovery. Those data train the VLA to handle similar states later. Whether the corrected episode succeeds and whether the trained VLA improves are different questions and require separate measurements.

Central hypothesis. Verified, executable GPT corrections collected on successive student-induced distributions improve a standalone VLA more than equal-budget offline demonstrations, text-only critique, or a single round of correction. The proposed paper must establish both that the teacher provides useful supervision and that the student acquires persistent capability. A hybrid system that succeeds only while GPT is available does not establish the latter.

Prior work and boundaries. DAgger, HG-DAgger, and SafeDAgger already establish iterative aggregation, expert intervention, and selective querying. Code as Policies already turns language reasoning into programs. The proposed increment is an auditable conversion of GPT-generated recovery programs into VLA-compatible supervision at learner-visited states, with teacher error and cost explicitly measured. GPT is an approximate expert; classical guarantees under stronger expert assumptions cannot simply be transferred to this setting.

2. Method: Executable Supervision on the Student Distribution

Actors and information. Let π_{θ_k} be the VLA at aggregation round k , E the GPT expert, and C the existing Code-as-Policy runtime. The expert receives the instruction, current permitted images, proprioception, and a bounded visible history h_t . It cannot read simulator target poses, hidden object state, success predicates, or future observations. Generated programs use a restricted set of declared sensing and skill calls; the same boundary applies to executed code, not just the prompt.

From correction to label. At a selected learner-visited state, $E(o_t, g, h_t)$ generates a correction program p_t . The runtime executes p_t from that actual state, recording the resulting sequence of observable histories and commands in the student’s action representation. A textual plan, intended waypoint, or VLA action merely judged “bad” is not an expert action label. For an action-chunk student, labels are contiguous executed expert segments with masks at intervention and handoff boundaries. Teacher future actions are never attached to observations from an incompatible student continuation.

Aggregation objective. Start with a declared demonstration dataset D_0 . Let D_k^E contain labeled corrective histories collected in round k and m_i mark executable, aligned label components. Optimize an appropriate supervised action loss, such as cloning or the student’s existing flow/diffusion objective:

$$\theta_{k+1} = \arg \min_{\theta} \left[\lambda_0 \mathbb{E}_{D_0} \ell_{\theta} + \sum_{j=1}^k \lambda_j \mathbb{E}_{D_j^E} [w_i m_i \ell_{\theta}(a_i^E \mid o_i, g, h_i)] \right].$$

The weights balance original skill retention, recent student states, and older corrections. Selection rules and weights are tuned only on development data. Unqueried student actions are not silently relabeled as expert demonstrations. GPT confidence alone is not a trustworthy label-quality score.

3. Three Correction Protocols within the Same Learning Loop

Approach A: dense state-conditioned correction. Query at predetermined action boundaries during learner rollouts and execute a short expert correction segment. This supplies a reference for the value of broad student-state coverage. Random query times with the same number of calls provide a budget-matched control. If the collector pauses simulation during GPT inference, it pauses at the queried state, records the pause, and charges wall-time cost; the result is a training protocol, not a real-time capability claim.

Approach B: bounded corrective takeover. A detector using visible progress, grasp evidence, or policy uncertainty requests GPT when the student appears to deviate. GPT temporarily takes over through a recovery program, then returns execution to the VLA after a declared observable condition or time limit. This is an intervention-based DAgger variant. Log the student-to-expert switch, expert segment, and handoff; longer teacher segments shift the collection distribution toward the expert and must be measured rather than described as purely student-generated experience.

Approach C: budgeted selective correction. A query rule estimates whether another expert label is useful from student uncertainty, novelty relative to the aggregated dataset, and expected correction benefit. Compare it with fixed-rate and randomly selected queries under the same call and execution budgets. A small randomized exploration quota samples apparently easy states and exposes blind spots. The teacher, label representation, and student-training procedure remain the same across protocols, isolating which states are queried.

4. Pipeline and the Correction-Data Contract

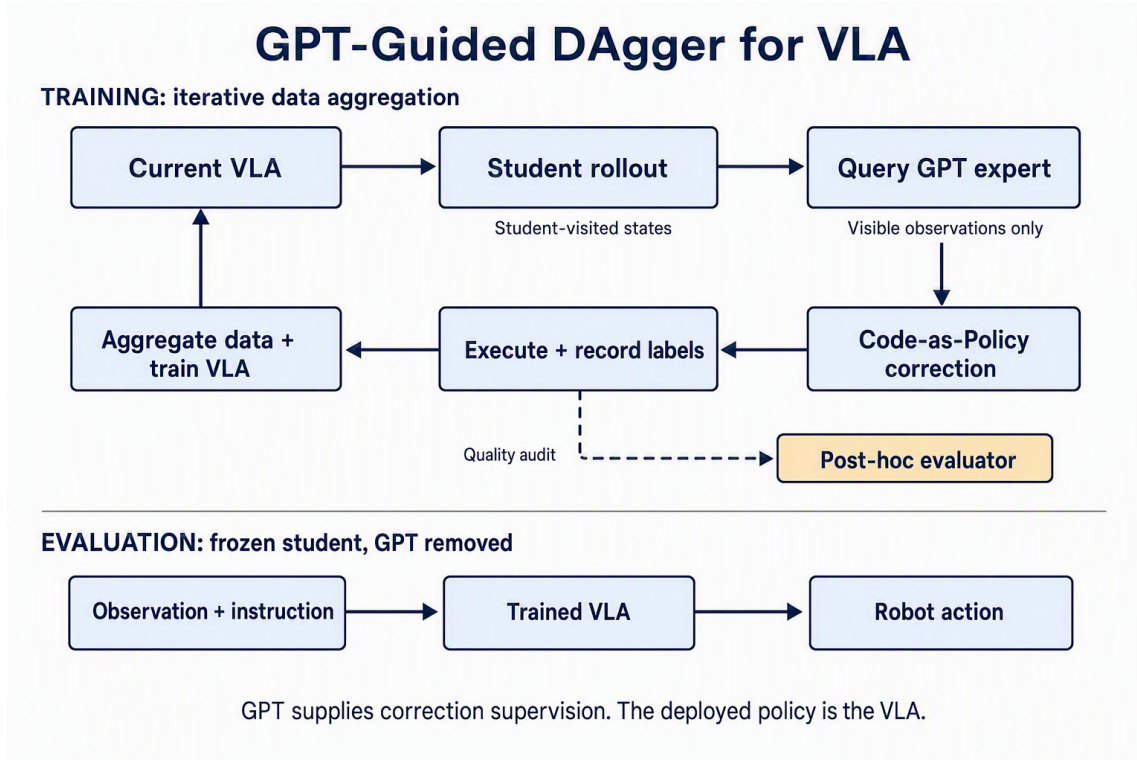


Figure 1: GPT supplies executable correction supervision during training. Student rollouts, corrective execution, aggregation, and retraining repeat across rounds. The lower band evaluates the frozen VLA with GPT removed. GPT-image-2 conceptual diagram. The post-hoc evaluator audits outcomes without exposing hidden state to the teacher.

Record structure. Each record binds task and round identifiers, student checkpoint, queried observation/history, expert program version, actual action segment, timestamps, control frame, action normalization, intervention boundaries, validity mask, and the later harness verdict. The evaluator’s hidden state stays outside teacher, student, Planner, Engineer, and Reviewer context. Post-hoc outcomes may support development-data audits and declared selection rules, never online access to hidden predicates.

What is validated. Check syntax and allowed calls, timestamp alignment, frame conversion, execution acknowledgements, and observable local effects. An eventual task success does not prove that every action was optimal. Conversely, an episode that later fails can contain a valid local correction. Retain failed and rejected traces, with reasons; compare quality filtering with all technically valid corrections to measure selection bias. Do not discard difficult states simply because the teacher failed to recover them.

Implementation boundary. The preferred collector executes corrections in the same environment state that triggered the query. Optional simulator branching uses evaluator-owned opaque checkpoints, exposes only permitted observations, and is reported separately. A simulated recovery from a modified or reset state cannot label the original state without an explicit alignment justification.

5. Experimental Design: Collection, Training, and Evaluation

Initial student and task separation. Start from one fixed VLA checkpoint and a declared D_0 , using existing Robot-RSI skill execution and trace collection. Select task families before collection, provisionally grouping 24 training instances, eight validation instances, and 16 held-out instances. Near-duplicate language or layout variants remain within one split; adjust counts rather than break family separation. Hold out object arrangements, physical perturbations, and designated task compositions, distinguishing unseen-task generalization from recovery on familiar tasks.

Aggregation schedule. An initial design uses three rounds, each with at most 200 collection episodes, 400 GPT queries, and a fixed expert-action duration cap. These are simultaneous ceilings, not guaranteed yields; report queries, accepted labels, expert control time, unique queried states, and completed verdicts. Freeze the GPT snapshot, prompt, runtime, and allowed skills across the main comparison. Each round deploys the newly trained student, while a fixed-student collection baseline isolates iterative distribution adaptation.

Policy mixing. The dense reference may use a declared expert/student mixing schedule, for example expert segment-selection probabilities of 0.5, 0.25, and 0 across rounds, while still querying student states when selected by the labeling protocol. The last round is student-driven except for explicitly logged corrective segments. Intervention-based methods report the realized expert-controlled fraction rather than claiming equivalence to canonical dense DAgger. All approaches must share an explicit collector protocol and budget.

Comparison	Additional data or assistance	Question isolated
Frozen initial VLA	None	Starting capability
More offline cloning	New expert rollouts from initial states	Data quantity versus visited-state relevance
One-round correction	One collected correction set	Iteration versus single fine-tuning
Fixed-student aggregation	Multiple rounds from the initial student	Value of updating the collection distribution
Text critique only	Critique, without executed expert targets	Whether language feedback supplies useful supervision
GPT-assisted hybrid	Expert remains available at evaluation	Online rescue versus learned student improvement
GPT-DAgger protocols	Dense, takeover, or selective correction	Query allocation and teacher supervision

Fair budgets. Match additional teacher execution time, accepted action-label count, and student gradient updates in separate controlled panels; one scalar rarely matches all three. Include a fixed-skill or human-expert correction baseline on a smaller matched subset to measure teacher quality. Text-only critique is a deliberately different feedback channel, not an equal-action-supervision expert. Hyperparameters, replay ratios, and checkpoint selection use validation only.

Frozen student tests. After every round, evaluate a frozen student without GPT on matched tasks and held-out tasks. A proposed held-out panel of 16 instances, four conditions, and five seeds yields 320 episodes per checkpoint. Use both natural initial-state rollouts and independently collected recovery-state probes; teacher-corrected training states are not held-out recovery tests. The standard 120-task LIBERO short panel separately reports task-level pass@10 with subset and suite breakdowns.

6. Planned Experiments and Evidence Required for the Claim

Experiment A: does correction become student capability? Compare the initial and round-1/2/3 students with GPT disabled, using matched seeds and held-out tasks. Plot simulator-confirmed success against cumulative labels, expert control time, GPT calls, and training compute. Report the GPT-assisted hybrid in a separate panel. Improvement confined to assisted execution supports an online rescue system, not successful DAgger-based transfer into the VLA. Training loss alone is insufficient.

Experiment B: why aggregate iteratively? Compare fresh student-induced collection with equal-size offline demonstrations, one-round correction, and repeated collection from the initial student. Keep student update counts and replay budgets controlled. Measure whether successive rounds cover new error states and reduce recurrence of previously corrected failure modes. If fixed-student or initial-state data achieve the same result, the on-distribution aggregation claim must be narrowed.

Experiment C: whether Code as Policy is the useful teacher interface. Hold queried states fixed and compare GPT text critique, direct numerical action proposals, fixed recovery skills, and executed GPT-generated programs. For same-state comparisons, use isolated evaluator-owned simulator branches where supported, with identical permitted observations and no state inspection by the agent. Report program validity, command execution, local correction success, label alignment, teacher wall time, and downstream student improvement. Extra runtime capability must not be confused with better reasoning.

Experiment D: teacher error and filtering. Compare all technically valid labels, declared observable-quality filtering, and post-hoc outcome-based filtering as distinct development-data variants. Include known alignment corruptions only as labeled diagnostic ablations. Measure teacher failure, label rejection by state difficulty, and the effect of retaining original demonstrations. A small human audit of permitted traces can estimate correction quality, but cannot silently repair one method’s training data. Successful-task filtering alone risks removing precisely the hard states DAgger is meant to cover.

Experiment E: query and intervention budgets. Compare random queries, fixed-rate queries, visible-failure triggers, and uncertainty/novelty selection. Match query counts and expert execution budgets. Vary takeover duration and replay weights to expose the tradeoff between obtaining a complete recovery and keeping data close to the student distribution. A selective method should improve learning per expert call without ignoring rare catastrophic or persistent failure modes.

Possible result	Supported conclusion
Standalone VLA improves on matched and held-out tasks	Supports persistent learning if teacher/data/compute controls also hold.
Hybrid improves but GPT-free student does not	Teacher rescue works; distillation or data alignment remains inadequate.
Equal-budget offline data match iterative correction	Student-distribution querying has not shown an independent advantage.
Selective queries match dense learning at lower cost	Supports efficient teacher allocation within the measured task distribution.
Aggressive filtering improves training metrics but harms hard tasks	Selection bias may remove difficult states; examine coverage and retention.

Primary endpoint and statistics. Preregister GPT-free held-out success improvement at a fixed expert budget. Use multiple student training seeds and paired evaluation seeds, with intervals clustered by task and variation across training runs reported separately. Validate sample size using development variance. Report forgetting on original tasks and inference efficiency alongside new recovery success.

7. Feasibility, Failure Modes, and Expected Paper Contribution

Robot-RSI integration. The VLA supplies student rollouts; existing Planner/Engineer capabilities generate and execute bounded correction programs; the trajectory store aggregates aligned action labels. Reviewer-visible evidence is restricted to allowed observations and receipts, while the independent harness supplies post-hoc task verdicts. Dataset construction records control ownership, teacher versions, and state provenance. Formal student evaluation disables teacher intervention and persistent write-back. No new force controller is required for this paper.

Partial observability and noisy expertise. GPT may misidentify objects, diagnose the wrong failure, or generate a recovery that is feasible but not useful. The student must receive the observable history needed to explain its labels. A teacher using privileged information unavailable to the student would create an unlearnable or unfair target; such access is excluded. Action ambiguity also means one valid correction need not be the unique optimal action. Model-appropriate sequence losses and multiple feasible trajectories may be preferable to treating every alternative as an error.

Latency and state drift. A delayed teacher response can refer to an obsolete state. A simulation collector may pause with cost accounting, while a physical collector must hold or resample safely and relabel from the actual observation at takeover. Never pair an old image with a correction executed after an unrecorded state change. This training-time issue is distinct from the real-time-control objective of the second paper.

Coverage, forgetting, and evidence. Corrections can overrepresent hard states or long teacher takeovers. Balance replay against D_0 , report the realized collection distribution, and evaluate retention. Preserve all rejected, failed, diagnostic, and interrupted records. Task success comes only from final simulator/harness verdicts; provider, transport, image, resource, and interrupted records are excluded from valid task denominators and reported separately, with implementation errors and missing coverage visible. Log teacher prompt/completion/total tokens, calls, unmetered calls, collection wall time, action/recovery time, and student training and inference cost.

Expected contribution. The paper should demonstrate a concrete learning interface between Code as Policy and VLA: GPT generates executable corrections, those corrections become aligned supervision on learner-visited states, and dataset aggregation improves a GPT-free student. Its evidence must isolate iteration, executable label quality, query allocation, and data scale. The novelty claim is this verified supervision mechanism and its measured learning value, not DAgger itself or continued reliance on GPT during deployment. A broader literature comparison is needed before any first-of-its-kind claim.

References

- [1] Ross, Gordon, and Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. 2011. <https://arxiv.org/abs/1011.0686>
- [2] HG-DAgger: Interactive Imitation Learning with Human Experts. 2018. <https://arxiv.org/abs/1810.02890>
- [3] Query-Efficient Imitation Learning for End-to-End Autonomous Driving. 2016. <https://arxiv.org/abs/1605.06450>
- [4] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [5] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [6] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>